Higher National School of Advanced Technologies (ENSTA)
SS/IA - First Semester 2026 - Module: WAMD

TD-TP 3
E-Commerce Backend

REST API with Node.js, Express.js, MongoDB, JWT and rate limiting

Student: OUINTNE Mohammed Amine
Group: AI G2
Repository: https://github.com/amouinten/first-backend-.git

1. Project overview

OuinShop Backend is the REST API for the e-commerce application built in TD-TP 2 (React + Redux). Products and users are stored in MongoDB instead of a local JavaScript array. The API lets clients browse products, register and log in, and it protects product creation, modification and deletion with JWT authentication. Login attempts and general API use are limited with rate limiting.

Technologies

Node.js, Express.js, MongoDB with Mongoose, bcryptjs (password hashing), jsonwebtoken (JWT), express-rate-limit, cors, helmet, dotenv. API tests were run with Thunder Client in Visual Studio Code.

Endpoints

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Register a new user	Public
POST	/api/auth/login	Log in and receive a JWT	Public
GET	/api/products	Retrieve all products	Public
GET	/api/products/:id	Retrieve one product	Public
POST	/api/products	Create a product	JWT required
PUT	/api/products/:id	Update a product	JWT required
DELETE	/api/products/:id	Delete a product	JWT required

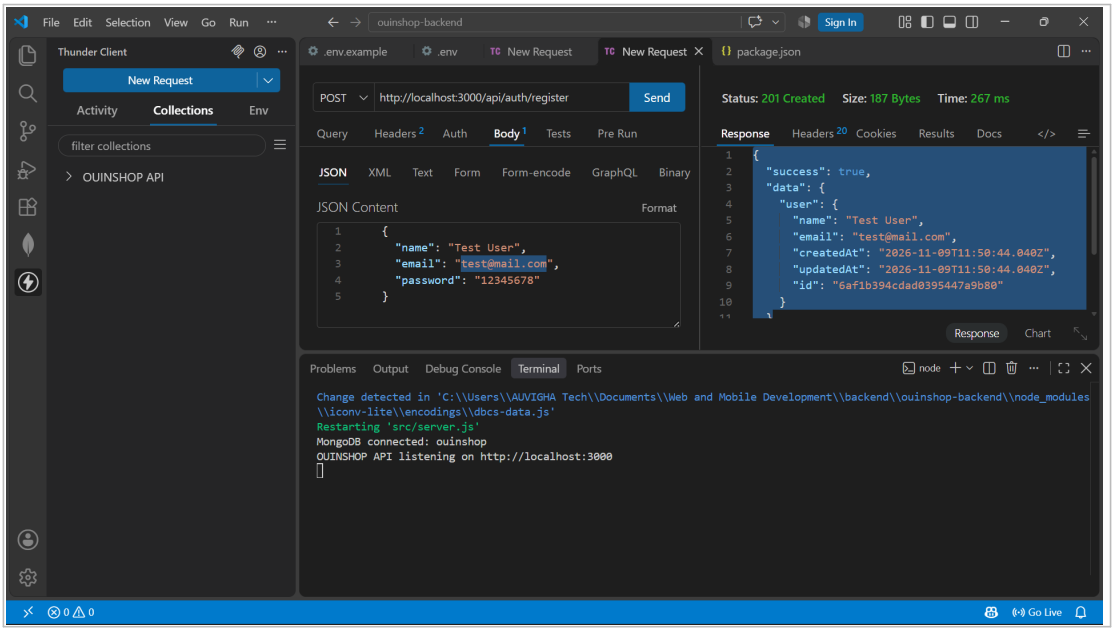
Summary of the tests shown in this report

Test	Request	Result
1. Register a new user	POST /api/auth/register	201 Created
2. Register with an existing email	POST /api/auth/register	409 Conflict
3. Login with valid credentials	POST /api/auth/login	200 OK with a JWT
4. Login with a wrong password	POST /api/auth/login	401 Unauthorized
5. Create a product (valid token)	POST /api/products	201 Created
6. Retrieve one product	GET /api/products/:id	200 OK
7. Delete a product (valid token)	DELETE /api/products/:id	200 OK
8. Retrieve the deleted product	GET /api/products/:id	404 Not Found
9. Rate limiting on login	POST /api/auth/login	429 Too Many Requests

2. Test results

1. Register a new user

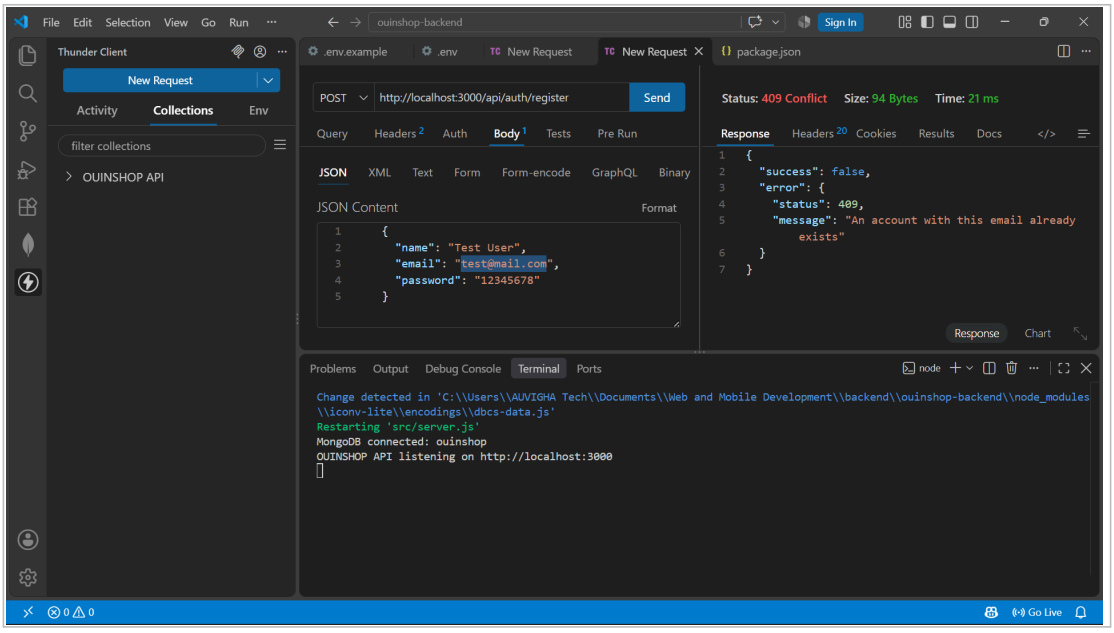
Request: POST /api/auth/register **Expected:** 201 Created



The user was created. The response contains the user's name, email, timestamps and id, and does not contain the password or its hash.

2. Register with an existing email

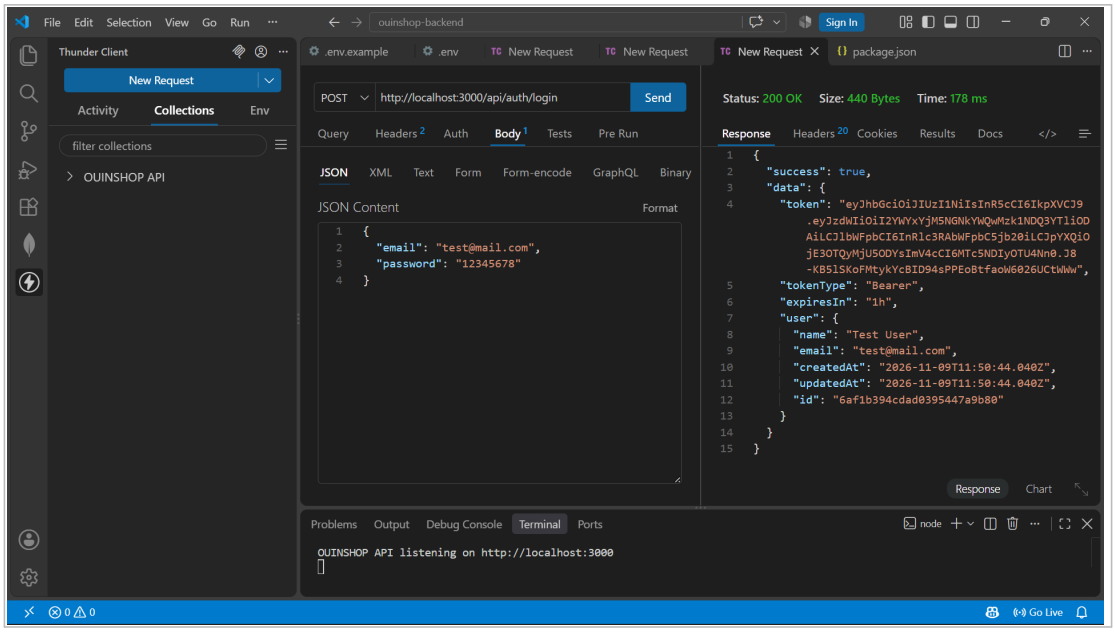
Request: POST /api/auth/register **Expected:** 409 Conflict



The duplicate email was rejected with the message "An account with this email already exists".

3. Login with valid credentials

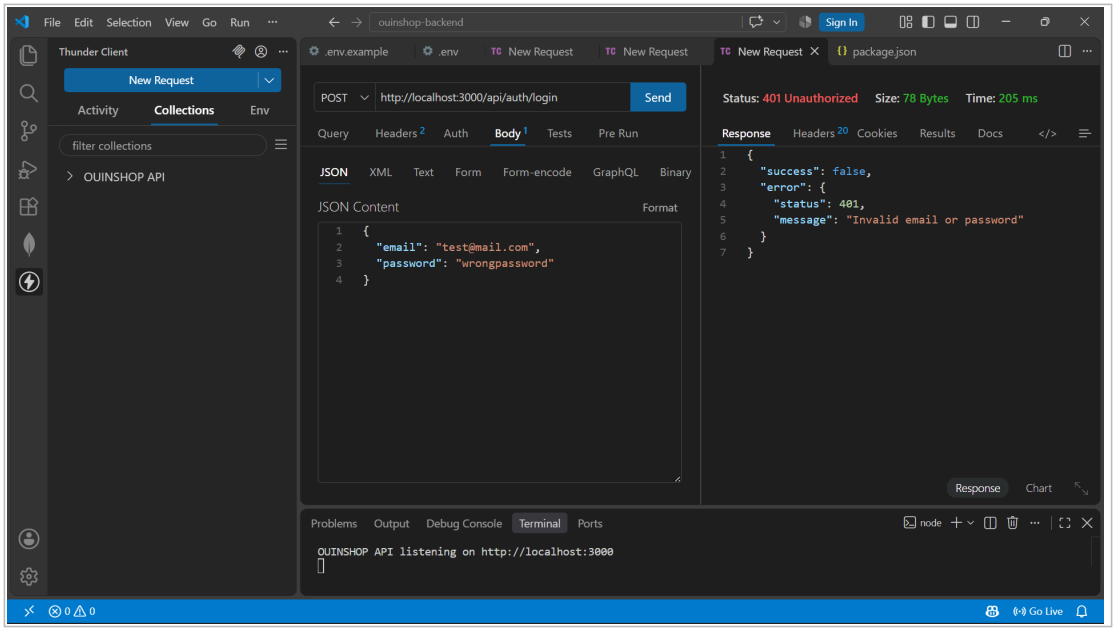
Request: POST /api/auth/login **Expected:** 200 OK with a JWT



The response contains a JWT, the token type (Bearer), its lifetime (1h) and the user's public information.

4. Login with a wrong password

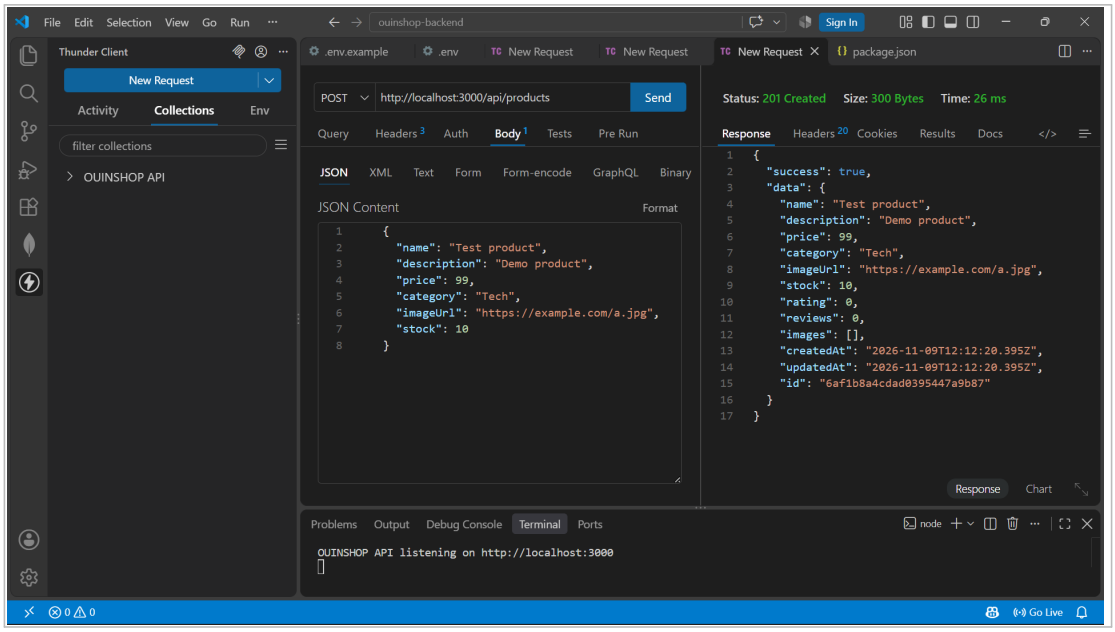
Request: POST /api/auth/login **Expected:** 401 Unauthorized



The login was rejected with the message "Invalid email or password". No token is returned.

5. Create a product (valid token)

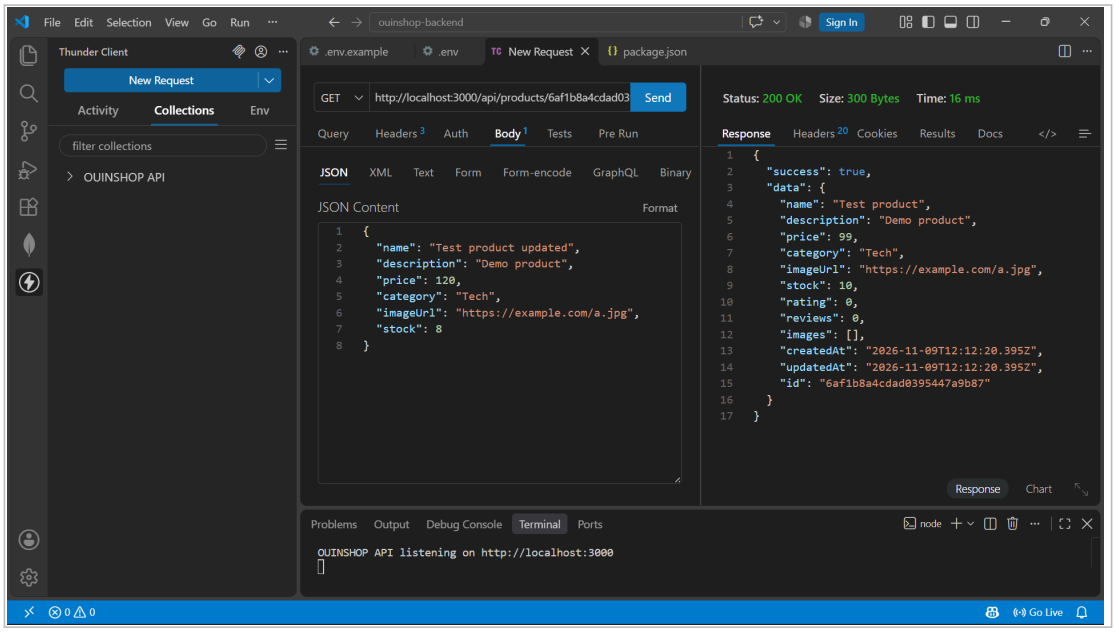
Request: POST /api/products **Expected:** 201 Created



The product was saved in MongoDB and returned with its generated id. The request was authorized with a Bearer token.

6. Retrieve one product

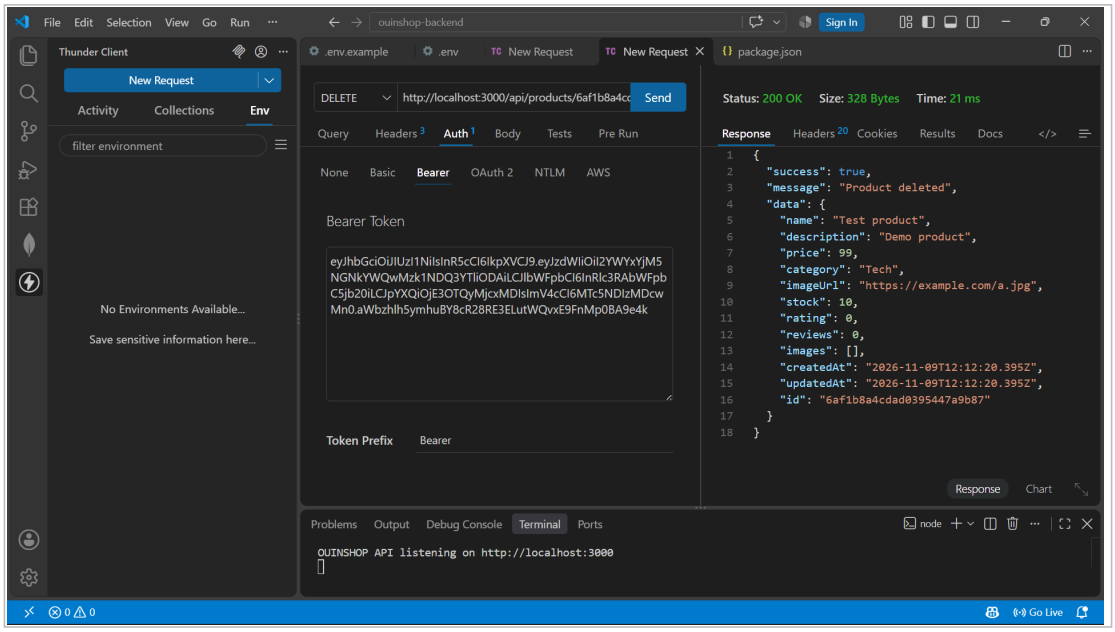
Request: GET /api/products/:id **Expected:** 200 OK



The product was retrieved by its id. This endpoint is public and requires no token.

7. Delete a product (valid token)

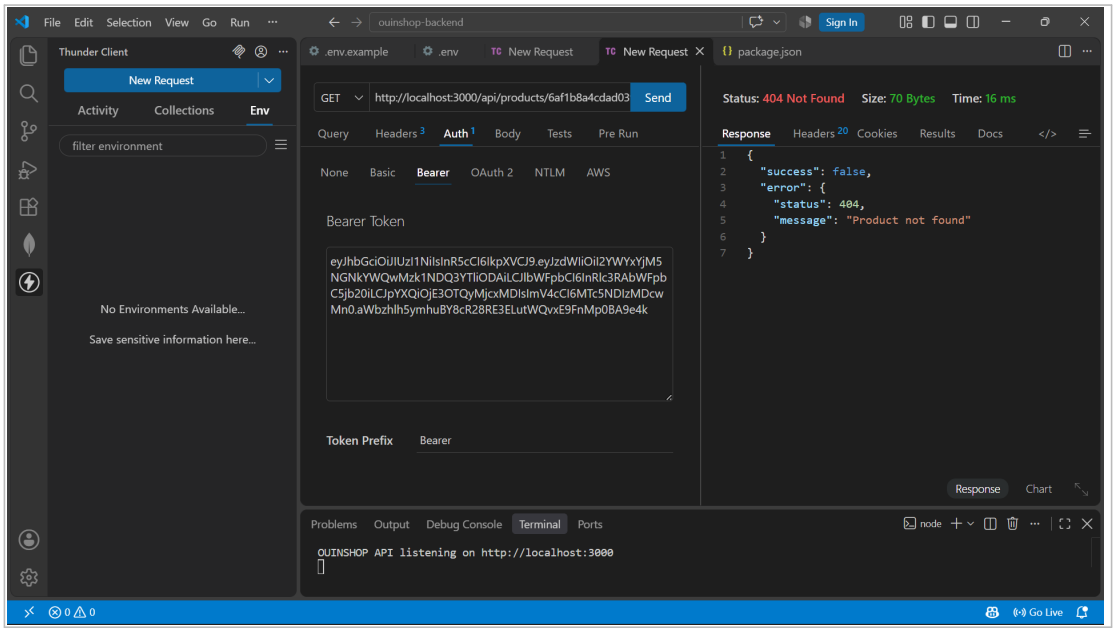
Request: DELETE /api/products/:id **Expected:** 200 OK



The product was deleted. The response contains the message "Product deleted" and the deleted document. A valid Bearer token was sent.

8. Retrieve the deleted product

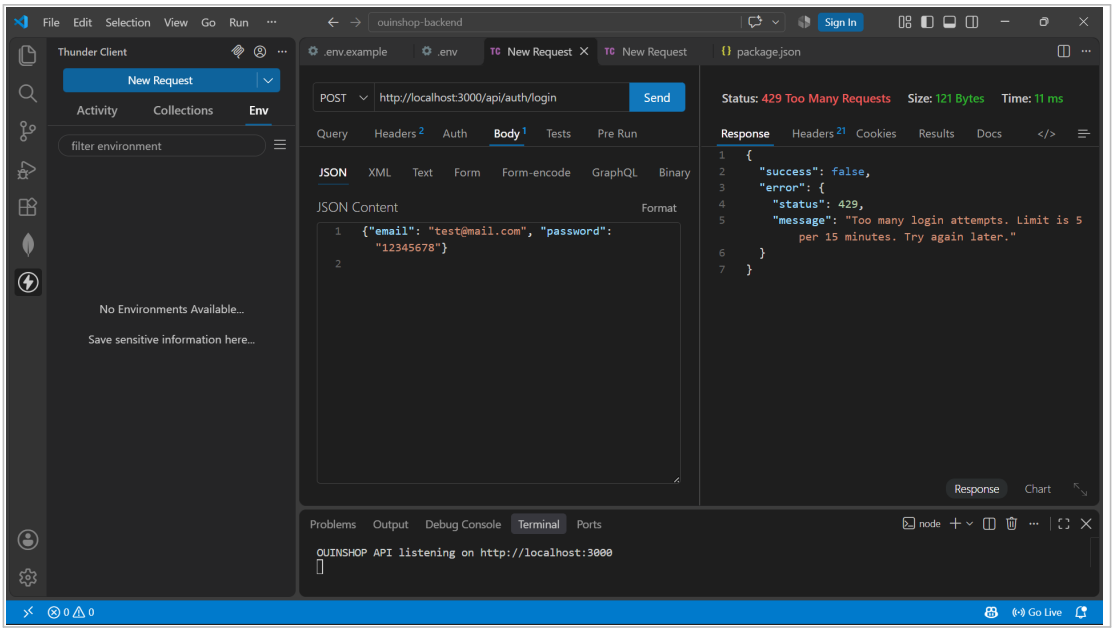
Request: GET /api/products/:id **Expected:** 404 Not Found



The deleted product can no longer be found. The error is returned in the API's standard format with the message "Product not found".

9. Rate limiting on login

Request: POST /api/auth/login **Expected:** 429 Too Many Requests



After 5 login requests within 15 minutes, the limiter blocked the request with the message "Too many login attempts. Limit is 5 per 15 minutes. Try again later."